Tema 02 — Floats

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 8

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: Representación de números reales

(a) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número `1000011,1101b`. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo, 7 bits de parte entera y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante con la siguiente distribución: 1 bit de signo de significante, 8 bits de significante, 1 bit de signo de exponente y 3 bits de exponente. ¿Cuál de estas representaciones recomendaría para almacenar el número?

(b) **[I2 2025-2]** Suponga que imprime en la consola de su computador un valor que se encuentra almacenado en formato float del estándar IEEE754. Al imprimir, en la consola ve que aparece el valor "`NaN`". ¿Es posible, a partir de este resultado, obtener la secuencia de 32 bits exacta que representa este valor? Justifique.

Pregunta 2: Punto fijo y Punto flotante

(a) Transforme los siguentes números a su notación en Punto Fijo (Signo/Magnitud) de 16 bits (parte racional de 8 bits).

- $-25{,}65625$
- $124{,}796875$

(b) Transforme los siguientes números decimales al estándar `IEEE-754` de 32 bits.

- 89,75
- 16,8125

Pregunta 3: Suma y multiplicacion en `IEEE-754`

(a) Realice la suma entre 89,75 y 16,8125 y represente el resultado en `IEEE-754`

(b) Interprete los números `0x40A00000` y `0x4170000` y multiplíquelos.

(c) Interprete los números `0x82D00000` y `0x7F800000` y multiplíquelos.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 8

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: Representación de números reales

(a) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número `1000011,1101b`. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo, 7 bits de parte entera y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante con la siguiente distribución: 1 bit de signo de significante, 8 bits de significante, 1 bit de signo de exponente y 3 bits de exponente. ¿Cuál de estas representaciones recomendaría para almacenar el número?

Solución: Primero, es útil ver ambas representaciones propuestas para el número.

- **Punto fijo =** $0_{\texttt{signo}}1000011_{\texttt{entero}}1101_{\texttt{fraccionario}}$
- **Punto flotante =** $0_{\texttt{ss}}10000111_{\texttt{s}}0_{\texttt{se}}110_{\texttt{e}}$

A partir de esto, es posible visualizar que en el caso de la representación de punto fijo no se pierden bits, mientras que en la representación de punto flotante se pierden los 3 bits menos significativos.
Por lo tanto, es recomendable utilizar la representación de punto fijo, dado que, al perder menos bits de información, es más preciso.

(b) **[I2 2025-2]** Suponga que imprime en la consola de su computador un valor que se encuentra almacenado en formato float del estándar IEEE754. Al imprimir, en la consola ve que aparece el valor "`NaN`". ¿Es posible, a partir de este resultado, obtener la secuencia de 32 bits exacta que representa este valor? Justifique.

Solución: No es posible. La representación del valor `NaN` del formato float tiene las siguientes características:

- Su exponente es igual a 11111111.

- **Al menos uno de sus bits de significante es distinto de cero.**

El último punto es relevante. Dado que cualquiera de los bits del significante es distinto de cero, al leer en consola el valor "`NaN`" no tenemos cómo saber cuál de los 23 bits es igual a 1.

Pregunta 2: Punto fijo y Punto flotante

(a) Transforme los siguentes números a su notación en Punto Fijo (Signo/Magnitud) de 16 bits (parte racional de 8 bits).

- $-25{,}65625$

Solución:

- **Signo [1 bit]** = `1b` (signo negativo)
- Parte entera $= 25 = 2^4 + 2^3 + 2^0 =$ `11001b` $\rightarrow$ (7 bits: `0011001b`)
- Parte fraccionaria $= 0.65625 = 2^{-1}+2^{-3}+2^{-5} =$ `0,10101b` $\rightarrow$ (8 bits: `10101000b`)

Por lo tanto, la representación en punto fijo es:

$$-25{,}65625 = \texttt{1 0011001 10101000}$$

- $124{,}796875$

Solución:

- **Signo [1 bit]** = `0b` (signo positivo)
- Parte entera $= 124 = 2^6 + 2^5 + 2^4 + 2^3 + 2^2 =$ `1111100b`
- Parte fraccionaria $= 0.796875 = 2^{-1} + 2^{-2} + 2^{-5} + 2^{-6} =$ `0,110011b` $\rightarrow$ (8 bits: `11001100b`)

Por lo tanto, la representación en punto fijo es:

$$124{,}796875 = \texttt{0 1111100 11001100}$$

(b) Transforme los siguientes números decimales al estándar `IEEE-754` de 32 bits.

- 89,75

Solución:

- **Signo [1 bit]** = `0b` (signo positivo)
- Parte entera $= 89 = 2^6 + 2^4 + 2^3 + 2^0 =$ `1011001b`
- Parte fraccionaria $= 0.75 = 2^{-1} + 2^{-2} =$ `0,11b`
- Notación científica = `1011001,11b` = `1,01100111b` $\cdot\ 2^6$
- **Significante [23 bits]** = `01100111000000000000000b`

- **Exponente [8 bits]** = `exp` $- 127 = 6 \Rightarrow$ `exp` $= 133 =$ `10000101b`

Por lo tanto, la representación en estándar `IEEE-754` es:

$$89,75 = \texttt{0 10000101 01100111000000000000000}$$

- 16,8125

Solución:

- **Signo [1 bit]** = `0b` (signo positivo)
- Parte entera $= 16 = 2^4 =$ `10000b`
- Parte fraccionaria $= 0.8125 = 2^{-1} + 2^{-2} + 2^{-4} =$ `0,1101b`
- Notación científica = `10000,1101b` = `1,00001101b` $\cdot\ 2^4$
- **Significante [23 bits]** = `00001101000000000000000b`
- **Exponente [8 bits]** = `exp` $- 127 = 4 \Rightarrow$ `exp` $= 131 =$ `10000011b`

Por lo tanto, la representación en estándar `IEEE-754` es:

$$16,8125 = \texttt{1 10000011 00001101000000000000000}$$

Pregunta 3: Suma y multiplicacion en `IEEE-754`

(a) Realice la suma entre 89,75 y 16,8125 y represente el resultado en `IEEE-754`

Solución: Recuerde que las representaciones de estos números en `IEEE-754` son:

- $89{,}75 = \texttt{0 10000101 01100111000000000000000} = \texttt{1,01100111b} \cdot 2^6$
- $16{,}8125 = \texttt{0 10000011 00001101000000000000000} = \texttt{1,00001101b} \cdot 2^4$

En el algoritmo de la suma en `IEEE-754` es requisito que los exponentes de cada número sean iguales. Para conseguir esto, se realizarán 2 *shift right* al número de menor exponente, de modo que la representación de -16,8125 ahora es:

$$16,8125 = \texttt{0,0100001101b} \cdot 2^6$$

Luego, el siguiente paso es sumar los significantes normalizados. En caso de que estos no tengan la misma cantidad de bits, se debe hacer *zero-padding*. Entonces, la suma es:

$$\begin{aligned} & 1,0110011100 \cdot 2^6 \\ + \; & 0,0100001101 \cdot 2^6 \\ = \; & 1,1010101001 \cdot 2^6 \end{aligned}$$

Sabemos que en el estándar `IEEE-754` el exponente se encuentra desfasado en 127. Entonces:

$$\texttt{exp} - 127 = 6 \Rightarrow \texttt{exp} = 133 = \texttt{10000101b}$$

Finalmente, viendo la resta entre los números es posible ver que el resultado será positivo. Por lo tanto, el resultado es:

$$0\ 10000101\ 10101010010000000000000$$

(b) Interprete los números `0x40A00000` y `0x4170000` y multiplíquelos.

Solución: Las representaciones de los números `0x40A00000` y `0x41700000` en el formato `IEEE-754` son:

- `0x40A00000` = `0 10000001 01000000000000000000000`
- `0x41700000` = `0 10000010 11100000000000000000000`

Para realizar la multiplicación, se hace uso del algoritmo visto en clases. En primer lugar, es posible ver que los exponentes no desfasados de los números son: 2 y 3 respectivamente,

por lo tanto, para obtener el nuevo exponente se utiliza la siguiente fórmula: $exp_a + exp_b +$ desfase:

$$2 + 3 + 127 = 132$$

Por lo tanto, se obtiene que el exponente es: 132 o `10000100b`. Luego, se deben multiplicar los significantes (es importante notar que para este cálculo se ignoran las comas):

```
   1,111 × 1,01
   1 111
  00 00x
 111 1xx
= 1001 011
```

Luego, la coma se desplaza tanto como indique la suma de las posiciones del multiplicando y el multiplicador, en este caso 5. Por lo tanto, el resultado de la multiplicación de los significantes es: `10,01011`. Entonces, el número resultante es: $\texttt{10,01011} \times 2^5$, el cual no está normalizado, por lo que se debe hacer un *shift left* para normalizarlo. Esto implica que el valor del exponente aumenta en 1, pasando de 132 a 133 o `10000101` en binario, mientras que el valor de la mantisa es: `1,001011`. Por lo tanto, el valor del número normalizado es: $\texttt{1,001011} \times 2^6$, por lo que su representación en el estándar `IEEE-754` es:

```
0 10000101 00101100000000000000000
```

El cual representa el valor 75, es decir, la multiplicación de 5×13

(c) Interprete los números `0x82D00000` y `0x7F800000` y multiplíquelos.

Solución: Las representaciones de los números `0x82D00000` y `0x7F800000` en el formato `IEEE-754` son:

- `0x82D00000` = `1 00000111 10100000000000000000000`
  - **Signo:** `1b` → negativo.
  - **Exponente:** 00000111 → $7 - 127 = -120$
  - **Significante:** `10100000000000000000000b` → `1,101b`
  - **Valor:** $-(\texttt{1,101} \cdot 10^{-120})\texttt{b}$

- `0x7F800000` = `0 11111111 00000000000000000000000`
  - **Signo**: `0b` → positivo.
  - **Exponente**: `10000000b` → 255 - 127 = 128.
  - **Significante**: `00000000000000000000000b` → `1,0b`.
  - **Valor**: $+\infty$

De lo anterior, se deduce que la operación a realizar es $+\infty \times -(\texttt{1,101} \cdot 10^{-120})\texttt{b} \times 2 = -\infty$, cuya representación en el estándar `IEEE-754` es `0xFF800000`.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: Floats

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: Representación de números Reales

1. Suponga que posee dos representaciones para números reales que desea utilizar para almacenar el número binario $0,00010111_b$.
   - **Representación 1 (Punto Fijo):** 1 bit de signo; 2 bits de parte entera; y 6 bits de parte fraccional.
   - **Representación 2 (Punto Flotante):** 1 bit de signo de significante; 4 bits de significante (sin bit implícito); 1 bit de signo de exponente; y 3 bits de exponente.

   ¿Cuál de estas representaciones presenta una menor pérdida de precisión para almacenar el número?
2. Suponga que se crea el tipo de dato **arquiFloat** de 32 bits a partir del nuevo estándar "IIC2343" para representar números de punto flotante. A diferencia del tipo de dato `float` del estándar IEEE754, este posee el siguiente formato:
   - Un bit de signo de significante.
   - Un bit de signo de exponente.
   - 10 bits de exponente no desfasado.
   - 20 bits de significante normalizado con bit implícito.

   Compare los dos estándares respondiendo las siguientes preguntas:

   a) Señale una ventaja y una desventaja del tipo de dato **arquiFloat** respecto del tipo de dato `float`.

   b) Señale cómo se representa el resultado de la operación $2^{22} + 7$ en formato `float` y **arquiFloat**. ¿Existe pérdida de precisión en alguno de los casos?

Solución:

1.1) Para este número, es recomendable hacer uso de la representación de **punto flotante**.

Su representación normalizada equivale a $1,0111 \times 10^{-4}$, por lo que, considerando el formato de 4 bits de significante sin bit implícito, se busca almacenar la secuencia fraccionaria 10111, por la cantidad e bits que tenemos para el significante. La secuencia resultante es: $\texttt{0}_{\texttt{Signo.sig}}\texttt{1011}_{\texttt{Significante}}\texttt{1}_{\texttt{Signo.exp}}\texttt{100}_{\texttt{Exp}}$.

De este resultado se deduce que **existe perdida de precisión de 1 bit**, ya que el significante disponible (4 bits) no es suficiente para representar el número normalizado (10111).

En cambio, en el formato de **punto fijo**, se intenta almacenar $0,00010111$. Dado que solo dispone de 6 bits para la parte fraccional, la secuencia almacenada es: $\texttt{0}_{\texttt{Signo}}\texttt{00}_{\texttt{Entero}}\texttt{000101}_{\texttt{Fracción}}$.

Se observa que en esta última representación se pierden los 2 bits menos significativos (11 al final de la cadena original), lo que equivale a una pérdida de precisión de $2^{-7} + 2^{-8}$. Por lo tanto, el formato de punto flotante es más preciso para este valor.

1.2.a) Una ventaja es que **arquiFloat** posee un rango mayor respecto a los números que puede representar dada su cantidad de bits de exponente. En este caso, el mayor exponente representable es $1111111110_b = 1022$ (si se asume la reserva para los valores infinitos y con bit de signo positivo), que es significativamente mayor al exponente máximo de `float` igual a 127. Por otra parte, una desventaja es la pérdida de precisión. En este caso, si incluimos el bit implícito, **arquiFloat** puede representar números de hasta 21 bits en comparación a `float` que puede representar números de hasta 24 bits.

1.2.b) El número que se busca representar, haciendo la suma entre 2^{22} y 7 con notación científica, es igual a $1,00000000000000000111 \times 10^{00010110}$. A continuación, veremos cómo se almacena en cada representación:

- **float:** El número es positivo (bit de signo igual a cero) y el exponente desfasado es igual a $22 + 127 = 149$. Por lo tanto, se almacena la secuencia $0_{\text{signo}}10010101_{\text{exp.}}00000000000000000001110_{\text{sig.}}$. No existe pérdida de precisión.
- **arquiFloat:** Tanto el número como su exponente son positivos (bits de signo iguales a cero). Por lo tanto, se almacena la secuencia:

$$0_{\text{signo}}0_{\text{signo exp.}}0000010110_{\text{exp.}}00000000000000000001_{\text{sig.}}$$

Se pierden dos bits de precisión, por lo que se pierde el valor exacto del número: se redondea a $2^{22} + 4$.

Pregunta 2: Conversión a `IEEE-754`

Transforme los siguientes números al formato IEEE-754 (32 bits):

1. 127
2. 3,140625
3. -1,6875
4. 0,1015625
5. -2,40625

Solución:

1. La representación binaria de 127 es `1111111b` = `1,111111b` $\times 2^6$. Por lo tanto, su exponente desfasado es $127 + 6 = 133 =$ `10000101b` y su bit de signo es `0`, dado que el número es positivo. La representación es:

   `0 10000101 11111100000000000000000`

2. Para $3,140625$, la parte entera es `11b` y la decimal es $0,140625 = 1/8 + 1/64 =$ `0,001001b`. El número completo es `11,001001b` = `1,1001001b` $\times 2^1$.

   - Signo: `0b` (positivo).
   - Exponente desfasado: $127 + 1 = 128 =$ `10000000b`.
   - Mantisa: `10010010000000000000000b`.

   La representación es:

   `0 10000000 10010010000000000000000`

3. Para $-1,6875$, la parte entera es `1b` y la decimal es $0,6875 = 1/2 + 1/8 + 1/16 =$ `0,1011b`. El número completo es `-1,1011b` $\times 2^0$.

   - Signo: `1b` (negativo).
   - Exponente desfasado: $127 + 0 = 127 =$ `01111111b`.
   - Mantisa: `10110000000000000000000b`.

   La representación es:

   `1 01111111 10110000000000000000000`

4. Para $0,1015625$, la parte entera es `0b` y la decimal es $1/16+1/32+1/128 =$ `0,0001101b`. Normalizado es `1,101b` $\times 2^{-4}$.

   - Signo: `0b` (positivo).
   - Exponente desfasado: $127 + (-4) = 123 =$ `01111011b`.
   - Mantisa: `10100000000000000000000b`.

   La representación es:

   `0 01111011 10100000000000000000000`

5. Para $-2,40625$, la parte entera es `10b` y la decimal es $0,40625 = 1/4 + 1/8 + 1/32 =$ `0,01101b`. El número completo es `-10,01101b` = `-1,001101b` $\times 2^1$.

   - Signo: `1b` (negativo).
   - Exponente desfasado: $127 + 1 = 128 =$ `10000000b`.
   - Mantisa: `00110100000000000000000b`.

   La representación es:

   `1 10000000 00110100000000000000000`

Pregunta 3: Conversión a decimal

Transforme los siguientes números de su representación hexadecimal a binario. Luego, interpretándolos con el estándar `IEEE-754`, indique su valor en decimal.

1. `0x42AA1000`
2. `0xC25A8000`
3. `0x7F90100B`
4. `0x41D48000`
5. `0xFF800000`
6. `0x45800180`

Solución:

Recuerde que, para la transformación de hexadecimal a binario, se debe reemplazar el valor de cada dígito hexadecimal a su representación binaria usando 4 bits

1. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `0 10000101 01010100001000000000000`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 1, por lo tanto, el número es negativo (-)
   - **Exponente:** El valor del exponente desfasado es `10000101b` = 133, esto implica que el exponente es: 133 - 127 = 6.
   - **Significante:** El valor del significante es `1,01010100001b`.

   Finalmente, se tiene que la representación del número es:

   $$\texttt{1,01010100001b} \times 2^6 = \texttt{1010101,00001b} = 85,03125$$

2. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `1 10000100 10110101000000000000000`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 1, por lo tanto, el número es negativo (-)
   - **Exponente:** El valor del exponente desfasado es `10000100b` = 132, esto implica que el exponente es: 132 - 127 = 5.
   - **Significante:** El valor del significante es `1,10110101b`.

   Finalmente, se tiene que la representación del número es:

   $$-\texttt{1,10110101b} \times 2^5 = -\texttt{110110,101b} = -54,625$$

3. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `0 11111111 00100000001000000001011`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 0, por lo tanto, el número es positivo (+)
   - **Exponente:** El valor del exponente desfasado es `11111111b`, entonces, necesariamente se trata de un caso especial.

- **Significante:** El valor del significante es `1,00100000001000000001011b`.

Dado que la parte fraccionaria del significante tiene al menos un bit igual a 1, se tiene que la representación del número es `NaN`.

4. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `0 10000011 10101001000000000000000`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 0, por lo tanto, el número es positivo (+)
   - **Exponente:** El valor del exponente desfasado es `10000011b` = 131, esto implica que el exponente es: 131 - 127 = 4.
   - **Significante:** El valor del significante es `1,10101001b`.

   Finalmente, se tiene que la representación del número es:

   $$\texttt{1,10101001b} \times 2^4 = \texttt{11010,1001b} = 26,5625$$

5. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `1 11111111 00000000000000000000000`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 1, por lo tanto, el número es negativo (-)
   - **Exponente:** El valor del exponente desfasado es `11111111b`, entonces, necesariamente se trata de un caso especial.
   - **Significante:** El valor del significante es `0b`.

   Dado que la parte fraccionaria del significante solo tiene bits iguales a 0 y el signo es negativo, se tiene que la representación del número es $-\infty$.

6. Al separar el valor en binario del número en signo, exponente y mantisa, según el estándar `IEEE-754`, se obtiene que:

   `0 10001011 00000000000000110000000`

   Con esto, es posible deducir que:

   - **Signo:** El valor del bit de signo es 0, por lo tanto, el número es positivo (+)
   - **Exponente:** El valor del exponente desfasado es `10001011b` = 139, esto implica que el exponente es: 139 - 127 = 12.
   - **Significante:** El valor del significante es `1,00000000000000011b`.

Finalmente, se tiene que la representación del número es:

$$\texttt{1,0000000000000011b} \times 2^{12} = \texttt{1000000000000,0011b} = 4096,1875$$

Pregunta 4: Suma y multipliacación de Floats

Realice las siguientes operaciones planteadas a continuación:

1. **(I1 2024-1)** Interprete los números A = `0x7F800000` y B = `0x40000000` y súmelos.
2. Interprete los números A = `0x42300000` y B = `0x41B80000` y súmelos.
3. Interprete los números A = `0x40200000` y B = `0x421B0000` y súmelos.
4. Multiplique los números 21,5 y 6 en el formato `IEEE-754`.
5. Multiplique los números 6,25 y -7,5 en el formato `IEEE-754`.
6. Interprete los números `0x430C0000` y `0x7fC00000` y multiplíquelos.

Solución:

1. Las representaciones de los números `0x7F800000` y `0x40000000` en binario son:
   - A = `0111111110000000 00000000000000000b`.
     - **Signo**: `0b` → positivo.
     - **Exponente**: `10000000b` → 255 - 127 = 128.
     - **Significante**: `00000000000000000000000b` → `1,0b`.
     - **Valor**: `+inf`
   - B = `0100000000000000 00000000000000000b`.
     - **Signo**: `0b` → positivo.
     - **Exponente**: `10000000b` → 128 - 127 = 1.
     - **Significante**: `00000000000000000000000b` → `1,0b`.
     - **Valor**: $(\texttt{1,0} \times \texttt{10}^{\texttt{1}})\texttt{b} = \texttt{1b} = 2$.

   De lo anterior, se deduce que la operación a realizar es igual a $\infty \times 2$, lo que en formato `float` se traduce como `+inf` = `0x7F800000`.

2. Las representaciones de los números `0x42300000` y `0x41b80000` en binario son:
   - A = `0x42300000` = `0 10000100 01100000000000000000000`
     - **Signo:** `0b` → Positivo.
     - **Exponente:** `10000100b` → $132 - 127 = 5$.
     - **Significante:** `01100000000000000000000b` → `1,011b`.
     - **Valor:** $\texttt{1,011b} \times 2^5$
   - B = `0x41B80000` = `0 10000011 01110000000000000000000`
     - **Signo:** `0b` → Positivo.
     - **Exponente:** `10000011b` → $131 - 127 = 4$.
     - **Significante:** `01110000000000000000000b` → `1,0111b`
     - **Valor:** $\texttt{1,0111b} \times 2^4$

   Al momento de aplicar el algoritmo de la suma los exponentes de ambos números deben ser iguales. Para esto, se debe hacer *shift right* al número con menor exponente, en este caso B. Por lo tanto, la representación de B queda como:

   $$B = \texttt{0,10111b} \times 2^5$$

   Ahora, con los exponentes igualados, es posible realizar la suma de los significantes:

   $$\begin{array}{rr} & \texttt{1,01100} \\ + & \texttt{0,10111} \\ = & \texttt{10,00011} \end{array}$$

   Por otro lado, el exponente del resultado es el mismo que el de los operandos (luego de igualar). Por lo tanto, el resultado de la suma es:

   $$\texttt{10,00011b} \times 2^5 = \texttt{1000011b} = 2^6 + 2^1 + 2^0 = 67$$

3. Las representaciones de los números `0x40200000` y `0x421B0000` en binario son:

   - A = `0x40200000` = `0 10000000 01000000000000000000000`
     - **Signo:** `0b` → Positivo.
     - **Exponente:** `10000000b` → $128 - 127 = 1$.
     - **Significante:** `01000000000000000000000b` → `1,01b`.
     - **Valor:** `1,01b` $\times 2^1$
   - B = `0x421B0000` = `0 10000100 00110110000000000000000`
     - **Signo:** `0b` → Positivo.
     - **Exponente:** `10000011b` → $132 - 127 = 5$.
     - **Significante:** `00110110000000000000000b` → `1,0011011b`
     - **Valor:** `1,0011011b` $\times 2^5$

   Para igualar los exponentes, se deben hacer 4 *shift right* a A. Por lo tanto, la representación de A queda como:

   $$A = \texttt{0,000101b} \times 2^5$$

   Ahora, con los exponentes igualados, es posible realizar la suma de los significantes:

   $$\begin{aligned} & \texttt{0,0001010} \\ + \ & \texttt{1,0011011} \\ = \ & \texttt{1,0100101} \end{aligned}$$

   Por otro lado, el exponente del resultado es el mismo que el de los operandos (luego de igualar). Por lo tanto, el resultado de la suma es:

   $$\texttt{1,0100101b} \times 2^5 = \texttt{101001,01b} = 2^5 + 2^3 + 2^0 + 2^{-2} = 41,25$$

4. Las representaciones de los números 21,5 y 6 en el formato `IEEE-754` son:

   - $21,5$ = `0 10000011 01011000000000000000000`
   - 6 = `0 10000001 10000000000000000000000`

   Para realizar la multiplicación, se hace uso del algoritmo visto en clases. En primer lugar, es posible observar que los exponentes no desfasados de los números son 4 y 2, respectivamente. Por lo tanto, para obtener el exponente del resultado se utiliza la siguiente fórmula: $exp_a + exp_b +$ desfase:

   $$4 + 2 + 127 = 133 = \texttt{10000101b}$$

   Luego, se deben multiplicar los significantes sin considerar las comas de los operandos:

   $$\begin{aligned} & \texttt{1,01011} \times \texttt{1,1} \\ & \texttt{1 01011} \\ & \texttt{10 1011} \\ = \ & \texttt{100 00001} \end{aligned}$$

Luego, la coma se desplaza tanto como indique la suma de las posiciones del multiplicando y el multiplicador, en este caso 6. Por lo tanto, el resultado de la multiplicación de los significantes es `10,000001b` y el número resultante es: `10,000001b` $\times 2^6$.

Dado que el número no se encuentra normalizado, es necesario hacer un *shift left*. Esto implica que el valor del exponente aumentará en 1, pasando a $134 =$ `10000110b`, mientras que el valor de la mantisa será `1,0000001b`.

Finalmente, con esta información es posible escribir la representación del resultado en el estándar `IEEE-754`, la cual es:

```
0 10000110 00000010000000000000000
```

El cual representa el valor 129, es decir, la multiplicación de $21,5 \times 6$.

5. Las representaciones de los números 6,25 y -7,5 en el formato `IEEE-754` son:

   - $6,25 =$ `0 10000001 10010000000000000000000`
   - $-7,5 =$ `1 10000001 11100000000000000000000`

   En primer lugar, es posible observar que los ambos exponentes no desfasados son 2. Por lo tanto, el exponente del resultado es:

   $$exp_a + exp_b + \text{desfase} = 2 + 2 + 127 = 131 = \texttt{10000011b}$$

   Luego, se deben multiplicar los significantes sin considerar las comas de los operandos:

   $$\begin{aligned} & \texttt{1,1001} \times \texttt{1,111} \\ & \texttt{1 1001} \\ & \texttt{11 001} \\ & \texttt{110 01} \\ & \texttt{1100 1} \\ = \ & \texttt{10111 0111} \end{aligned}$$

   Luego, la coma se desplaza tanto como indique la suma de las posiciones del multiplicando y el multiplicador, en este caso 7. Por lo tanto, el resultado de la multiplicación de los significantes es `10,1110111b` y el número resultante es: `10,1110111b` $\times 2^4$.

   Dado que el número no se encuentra normalizado, es necesario hacer un *shift left*. Esto implica que el valor del exponente aumentará en 1, pasando a $132 =$ `10000100b`, mientras que el valor de la mantisa será `1,01110111b`. Es importante notar que, a partir de los signos del operando, el resultado de la multiplicación será negativo.

   Finalmente, con esta información es posible escribir la representación del resultado en el estándar `IEEE-754`, la cual es:

   ```
   1 10000100 01110111000000000000000
   ```

   El cual representa el valor -46.875, es decir, la multiplicación de $6,25 \times -7,5$.

6. Las representaciones de los números `0x430c0000` y `0x7fc00000` en binario son:

   - `0x430c0000` = `0 10000110 00011000000000000000000`
   - `0x7fc00000` = `0 11111111 10000000000000000000000`

   Si interpretamos los números con el estándar, es posible observar que corresponden a: 140 y `NaN` respectivamente, con ello es posible determinar que no se podrían operar los valores.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2024)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (1.5 ptos.) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número 101,10011b. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo; 3 bits de parte entera; y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante **sin bit implícito** con la siguiente distribución: 1 bit de signo de significante; 4 bits de significante normalizado; 1 bit de signo de exponente; y 2 bits de exponente. ¿Cuál de estas representaciones recomendaría para almacenar el número? Justifique su respuesta en base a la precisión de la representación.

Solución: Para este número, es recomendable hacer uso de la representación de **punto fijo**. A través de ella, el número se almacena como: $0_{\text{Signo}}101_{\text{Entero}}1001_{\text{Signo}}$, donde se deduce que se pierde el bit menos significativo de la parte fraccional. En cambio, en el formato de punto flotante, su representación normalizada equivale a $1,0110011 \times 10^{10}$, por lo que se almacena la siguiente secuencia: $0_{\text{Signo sig.}}1011_{\text{Significante}}0_{\text{Signo exp.}}10_{\text{Exponente}}$. Se observa que en esta última representación se pierden los 4 bits menos significativos, por lo que su resultado es menos preciso. Se otorgan **0.75 ptos.** por indicar la representación que implica menor pérdida de precisión; y **0.75 ptos.** por justificar indicando la cantidad de bits perdidos en cada caso.

(b) (1.5 ptos.) Suponga que se quiere utilizar el computador básico visto en el curso para realizar operaciones sobre **números reales** bajo la representación de **punto fijo**. Para ello, los tres bits más significativos de los registros `A`, `B` se reservan para la parte entera; mientras que los cinco bits restantes se utilizan para la parte fraccional. Considerando que **no se realizan modificaciones de *hardware***, ¿existirá algún problema al momento de realizar operaciones con la ALU? Justifique su respuesta solo aludiendo a las operaciones aritméticas (`ADD`, `SUB`, `SHL`, `SHR`) ejecutadas en la arquitectura.

Solución: Si se opera con registros que almacenan números representados por punto fijo con **la misma cantidad de bits fraccionales** (en este caso, 5), entonces la suma y resta se pueden expresar como: $A \pm B = (A' \times 10^{-5} \pm B' \times 10^{-5}) = (A' \pm B') \times 10^{-5}$. Es decir, podemos computar de forma consistente la suma y la resta de los registros, aplicando solo el punto decimal al momento de representar el resultado. Por otra parte, las operaciones de *shifting* seguirán funcionando como la multiplicación o división por 2, solo que manteniendo una cantidad constante de puntos decimales. Por lo tanto, **no existe problema en operar con esta representación a nivel de *hardware***. Se otorgan **0.75 ptos.** por señalar correctamente que no existen problemas en términos aritméticos; y **0.75 ptos.** por justificación. En este último criterio, se aceptan ejemplos para mostrar que los resultados no varían en esta representación.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar si la representación en formato float del resultado presenta cambios en el valor obtenido.

- (1.5 ptos.) $A + B$. A = 0x4A800000, B = 0x41700000.

Solución:

- A = 01001010100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10010101b → 149 - 127 = 22.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^{10110})$b = 4194304.
- B = 01000001011100000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000010b → 130 - 127 = 3.
  - **Significante:** 11100000000000000000000b → 1,111b.
  - **Valor:** $(1,111 \times 10^{11})$b = 1111b = 15.

De lo anterior, se deduce que la operación a realizar es 4194304+15. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,00000000000000000111 \times 10^{10110}$. Luego, sumando los significantes, se tiene $A + B = 1,00000000000000000111 \times 10^{10110}$. Finalmente, si se almacena como `float`, se tiene el valor 01001010100000000000000000011110b = 0x4A80001E = 4194319. **No existe pérdida de precisión**.
Se otorgan **0.375 ptos.** por la desagregación e interpretación correcta de cada número; y **0.75 ptos.** por señalar correctamente el resultado de la representación en formato `float`.

- (1.5 ptos.) $A \times B$. A = 0x7F800000, B = 0x40000000.

Solución:

- A = 01111111100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 11111111b → 255 - 127 = 128.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** + ínf.
- B = 01000000000000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $1,0 \times 10^1 = 10\text{b} = 2$.

De lo anterior, se deduce que la operación a realizar es igual a $+\infty \times 2$, lo que en formato float se traduce como $+\infty$ = 0x7F800000. Se otorgan **0.375 ptos.** por la desagregación e interpretación correcta de cada número; y **0.75 ptos.** por señalar correctamente el resultado de la representación en formato `float`.

Pregunta 2: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Explique el significado y uso del identificador que envía el controlador de interrupciones luego de recibir la señal INTA por parte de la CPU.

Solución: Al recibir la señal INTA, la CPU espera obtener el identificador asociado al **dispositivo cuya interrupción será atendida**. Este será enviado por el controlador de interrupciones. Luego, la CPU hace uso de este valor como índice del vector de interrupciones para obtener el puntero de la ISR del dispositivo atendido, permitiendo así la ejecución de su subrutina. Se otorgan **0.5 ptos.** por indicar la asociación entre el identificador y los dispositivos I/O; y **0.5 ptos.** por indicar su uso en la ejecución de la ISR deseada.

(b) (1 pto.) Indique cómo se lleva a cabo la transferencia de datos entre dispositivos I/O y la memoria RAM en arquitecturas que **carecen** de un controlador de DMA.

Solución: En este caso se lleva a cabo *Programmed* I/O; *i.e.* la transferencia manual de los datos hacia los registros de la CPU, los que luego son transferidos a destino (ya sea la RAM o dispositivos I/O de almacenamiento). Se otorga **1 pto.** si se describe correctamente del rol de la CPU en este tipo de interacción. No se otorga puntaje parcial.

(c) (4 ptos.) Para evitar que las plantas de interior de su hogar se sequen, usted implementa un sistema de regado automatizado conectado a un mini-computador con ISA RISC-V. A cada una de sus plantas le añade un sensor de humedad y un regador, cuyos registros de 32 bits son mapeados en memoria desde la dirección 0xABADBABE:

Offset	Nombre	Descripción
0x00	`humidity_sensor_cactus`	Sensor humedad cactus
0x04	`humidity_sensor_monstera`	Sensor humedad monstera
0x08	`irrigator_status_cactus`	Registro estado regador cactus
0x0C	`irrigator_status_monstera`	Registro estado regador monstera
0x10	`irrigator_command_cactus`	Registro comando regador cactus
0x14	`irrigator_command_monstera`	Registro comando regador monstera

Nombre	Descripción	Valor
`humidity_sensor_*`	Porcentaje humedad	0-100
`irrigator_status_*`	Apagado	0
`irrigator_status_*`	Encendido	1
`irrigator_status_*`	Riego suave	2
`irrigator_status_*`	Riego medio	4
`irrigator_status_*`	Riego alto	8
`irrigator_command_*`	Encender	1
`irrigator_command_*`	Apagar	255
`irrigator_command_*`	Detener riego	128
`irrigator_command_*`	Regar (por defecto suave)	2
`irrigator_command_*`	Incrementar riego	3
`irrigator_command_*`	Decrementar riego	4

Luego de varias pruebas, usted desarrolla el siguiente programa para llevar a cabo el riego:

```
irrigation_system:
  li t0, 0xABADBABE            # Carga en el registro t0 el valor del literal 0xABADBABE
  li s0, 15
  li s1, 20
  li s2, 0
  li s3, 1
  step_one:
    lw t1, 8(t0)
    beq t1, s2, step_two
    bne t1, s3, step_three
    beq zero, zero, step_four                # Salto incondicional a step_four
  step_two:
    li t2, 1
    sw t2, 16(t0)
    beq zero, zero, step_four                # Salto incondicional a step_four
  step_three:
    li t2, 128
    sw t2, 16(t0)
  step_four:
    li t2, 2
    sw t2, 16(t0)
    li t2, 3
    sw t2, 16(t0)
    while_one:
        lw t3, 0(t0)
        blt t3, s0, while_one
    li t2, 4
    sw t2, 16(t0)
    while_two:
        lw t3, 0(t0)
        blt t3, s1, while_two
    li t2, 128
    sw t2, 16(t0)
    li t2, 255
    sw t2, 16(t0)
  end:                         # Término del programa.
```

En base a este programa, conteste las siguientes preguntas:

1. (1 pto.) ¿Qué modo de comunicación existe entre el computador y el sistema de riego?

Solución: El modo de comunicación existente consiste en *polling*, dado que en el *label* `step_four` se evidencia la consulta constante sobre el valor del sensor de humedad para gestionar el riego de la planta. No obstante lo anterior, también se aceptan las siguientes respuestas:

- Comunicación en base a interrupciones, si se justifica que el código anterior consiste en la ISR ejecutada por el computador para llevar a cabo el riego de una planta.
- *Memory mapped* I/O. Si bien esto alude a la forma en la que se accede a los datos de los registros asociados a los dispositivos, dada la ambigüedad del enunciado se acepta también como respuesta **siempre** que se señale como argumento una de dos opciones: (1) la única forma de acceso a dispositivos a RISC-V; (2) el acceso a los registros de los dispositivos con instrucciones de lectura y escritura de memoria.

En cualquiera de los tres casos, se otorga **1 pto.** siempre y cuando **exista una justificación donde se apliquen correctamente los conceptos de I/O**. No se otorgará puntaje parcial.

2. (3 ptos.) Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por línea de código.

Solución:

- `step_one`: Se revisa el estado del regador del cactus. Si está apagado, se ejecuta el código del *label* `step_two`. Si está regando, independiente de la intensidad, se ejecuta el código del *label* `step_three`. En cualquier otro caso (*i.e.* está encendido), se ejecuta el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores.

- `step_two`: Se enciende el regador del cactus y se procede a ejecutar el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores.

- `step_three`: Se detiene el riego del regador del cactus y se procede a ejecutar el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores. En este caso, no es necesario explicitar la ejecución de `step_four`.

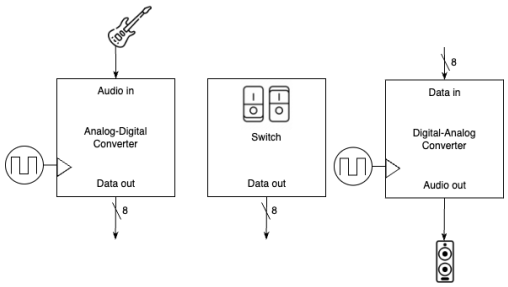
- `step_four`: Se activa el riego del regador del cactus y se incrementa para que realice riego medio. Luego, en `while_one` se consulta el valor del sensor de humedad del cactus constantemente hasta que sea mayor o igual a un 15 %. Posteriormente, se decrementa el nivel de riego para que tenga una intensidad suave. Finalmente, en `while_two` se consulta el valor del sensor de humedad del cactus hasta que sea mayor o igual a un 20 %, momento en el que se detiene el regador y se apaga. Se otorgan **1.5 ptos.** si se describe correctamente; y se descuentan **0.5 ptos.** por cada detalle faltante en la descripción.

Importante: Se considera como error en cada *step* si se toma en cuenta la monstera: en el código nunca se accede a los dispositivos conectados a ella.

Pregunta 3: Computador básico e I/O (6 ptos.)

Dado su interés por la música y la tecnología, ha empezado a explorar el funcionamiento de los pedales de efecto haciendo uso del computador básico para modificar las señales de sonido emitidas por una guitarra. Para esto, habilita la conexión con los siguientes dispositivos *port-mapped*:

Dispositivo	Tipo	Puerto
Conversor análogo-digital	Entrada	0
Switch de efecto	Entrada	1
Conversor digital-análogo	Salida	2



El conversor análogo-digital envía la digitalización de la señal generada por la guitarra; mientras que el conversor digital-análogo recibe la señal digital con efectos aplicados por el computador, los que son finalmente transmitidos a un amplificador. Adicionalmente, añade un *switch* con señal de salida de 8 bits, pero solo con dos valores posibles: 0 si está apagado y 1 si está prendido. El efecto del pedal se aplica solo si el *switch* está encendido. Para lograr este comportamiento, debe modificar la microarquitectura del computador básico implementando las siguientes instrucciones:

Instrucción	Operandos	Operación
`IN`	`A,Lit`	`A` = Dispositivo asociado a puerto `Lit`
	`B,Lit`	`B` = Dispositivo asociado a puerto `Lit`
`OUT`	`Lit,A`	Dispositivo asociado a puerto `Lit` = `A`
	`Lit,B`	Dispositivo asociado a puerto `Lit` = `B`

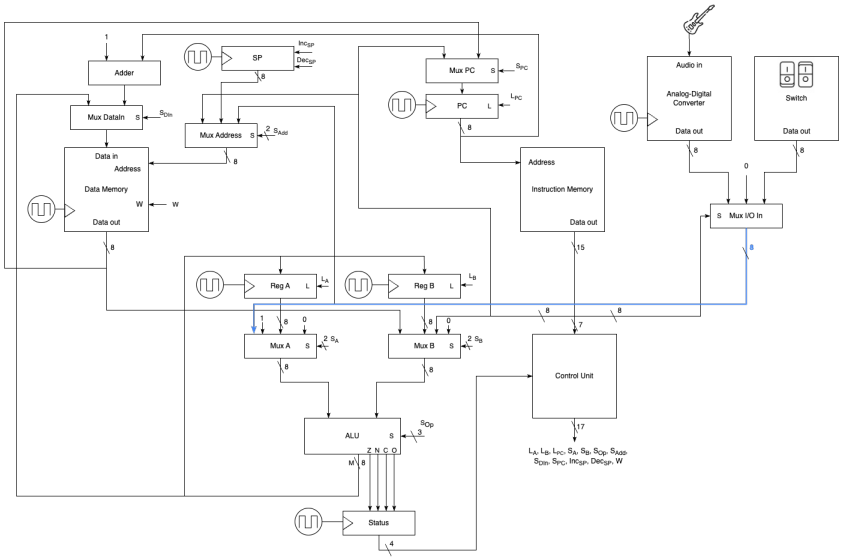
Para cada instrucción nueva, deberá incluir la combinación **completa** de señales que la ejecutan. Por cada señal de carga/escritura/incremento/decremento, deberá indicar si se activan (1) o no (0); en las señales de selección, deberá indicar el **nombre** de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama. Para el conversor digital-análogo, asuma que si no se le envían datos mediante `OUT`, debe recibir un 0 por defecto.

(a) (2.5 ptos.) `IN Reg, Lit`. Guarda en `Reg` (`A`/`B`) el valor de salida del puerto `Lit`.

Solución: A continuación, se describe una posible implementación:

- Se agrega el multiplexor "`Mux I/O In`" con una señal de selección de 8 bits para escoger uno de varios dispositivos I/O. Si bien solo se conectarán dos, es necesario considerar que el puerto de 8 bits requiere funcionar con valores exactos. Para los puertos no definidos, se selecciona una señal constante igual a cero. Para la selección, se utiliza el literal proveniente de la memoria de instrucciones.
- Se conecta la salida del multiplexor anterior con la entrada libre del multiplexor "`Mux A`", lo que permite almacenar el valor de los dispositivos en cualquiera de los registros.

El siguiente diagrama muestra la implementación anterior:



Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`IN A, Lit`	1	0	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-
`IN B, Lit`	0	1	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-

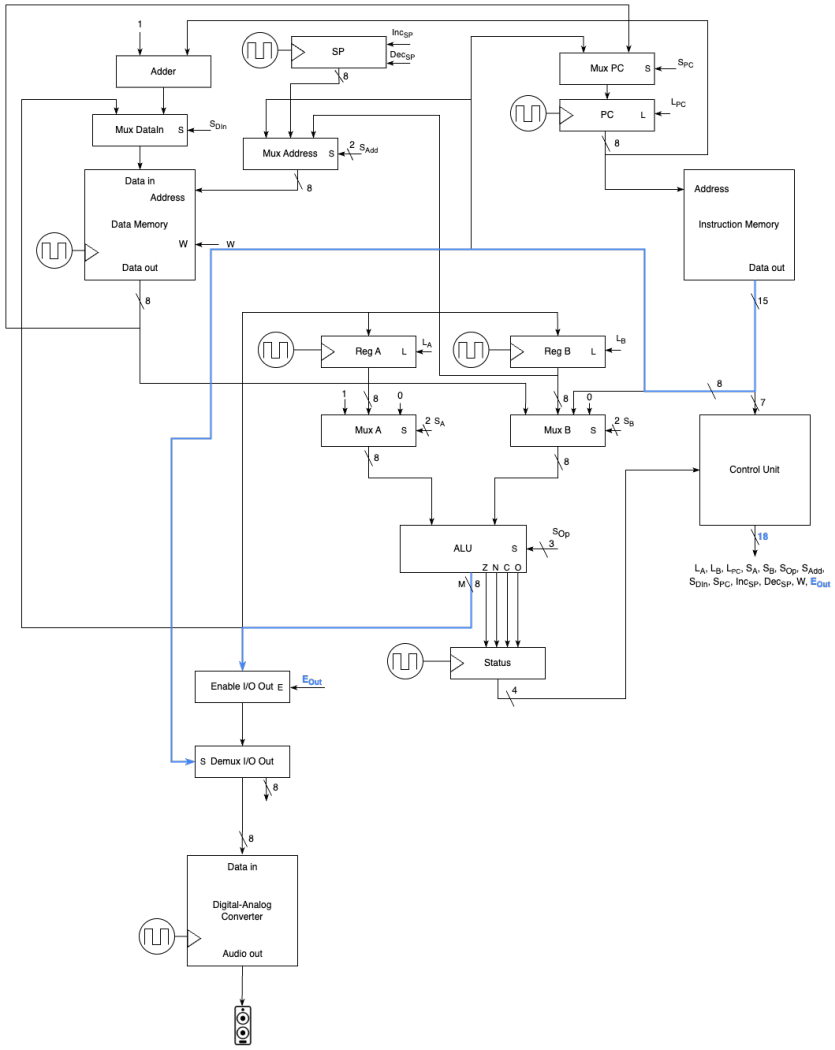
Se otorgan **1.25 ptos.** por la correctitud de la modificación y **1.25 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

(b) (2.5 ptos.) `OUT Lit, Reg`. Entrega como entrada al puerto `Lit` el valor de `Reg` (`A`/`B`).

Solución: A continuación, se describe una posible implementación:

- Se agrega el demultiplexor "`Demux I/O Out`" con una señal de selección de 8 bits para enviar el valor de entrada a uno de varios dispositivos I/O. Si bien solo habrá un dispositivo disponible para el envío, es necesario considerar que el puerto de 8 bits requiere funcionar con valores exactos. Para los puertos no definidos, simplemente se envía la señal a cables no conectados a otros dispositivos.
- La salida de la ALU se conecta con un *enabler* "`Enable I/O Out`", con señal habilitadora E_{Out} solo activa para las instrucciones de tipo `OUT`. Finalmente, su salida se conecta con la entrada del demultiplexor anterior para el envío correcto del valor de los registros `A` o `B`.

El siguiente diagrama muestra la implementación anterior:

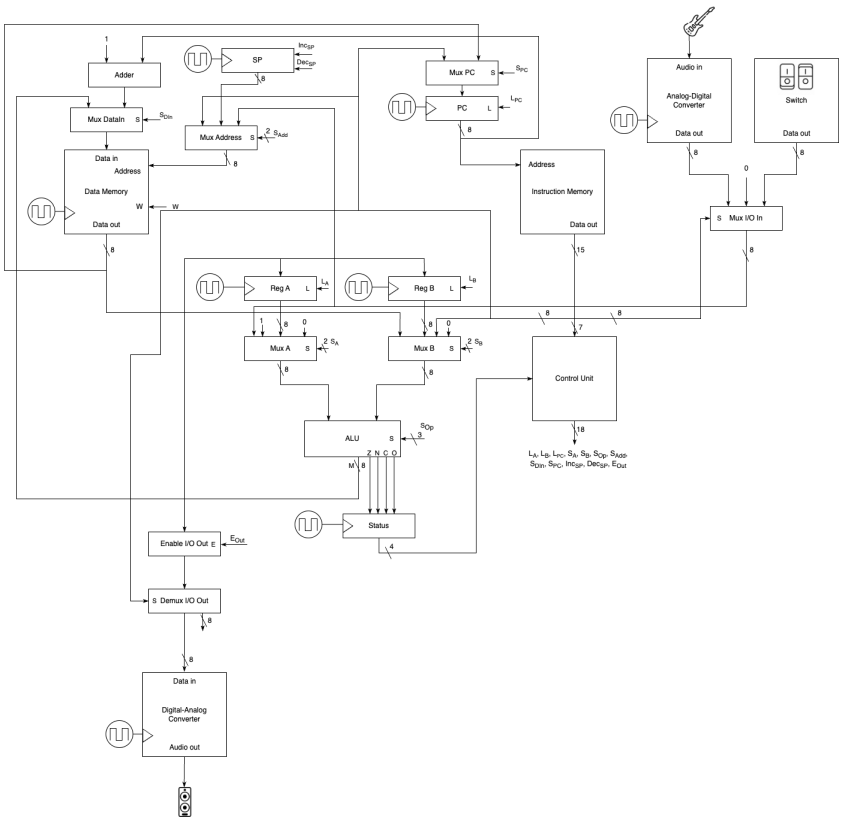


Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	E_{Out}
`OUT Lit, A`	0	0	0	0	0	0	A	ZERO	ADD	-	-	-	1
`OUT Lit, B`	0	0	0	0	0	0	ZERO	B	ADD	-	-	-	1

Se otorgan **1.25 ptos.** por la correctitud de la modificación y **1.25 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

A modo de completitud, se comparte a continuación el diagrama completo con todas las instrucciones solicitadas.



Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	E_{Out}
`IN A, Lit`	1	0	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-	0
`IN B, Lit`	0	1	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-	0
`OUT Lit, A`	0	0	0	0	0	0	`A`	`ZERO`	`ADD`	-	-	-	1
`OUT Lit, B`	0	0	0	0	0	0	`ZERO`	`B`	`ADD`	-	-	-	1

(c) (1 pto.) Asuma que se implementan las instrucciones anteriores. Elabore un fragmento de código Assembly que aplique el efecto del pedal a través del llamado a una subrutina `apply_effect`. Esta asumirá que la señal de sonido se encontrará almacenada en `B` y modificará su valor directamente. El programa debe, entonces:

1. Obtener el valor de la señal del conversor análogo-digital y el estado del *switch*.
2. Aplicar el efecto a partir del llamado a `apply_effect` **solo** si el *switch* está activado.
3. Enviar al conversor digital-análogo la señal resultante.

Debe hacer uso de las instrucciones `IN` y `OUT` para cumplir lo anterior. Puede agregar instrucciones adicionales ya existentes en la ISA del computador básico para completar el programa. **Solo debe llamar a `apply_effect`**, no es necesario que defina su código.

Solución: A continuación, se muestra un fragmento de código que cumple lo solicitado.

```
CODE:
  ; Paso 1: Obtención de la señal del conversor ADC y el switch.
  IN A,1
  IN B,0
  ; Paso 2: Aplicación del efecto solo si el switch está activo.
  CMP A,1
  JNE end
  CALL apply_effect
  ; Paso 3: Envío de la señal resultante a través de B, independiente
  ; de la aplicación del efecto.
  end:
    OUT 2,B
```

Se otorgan **0.25 ptos.** por obtener correctamente el valor de las señales de los dispositivos de entrada; **0.5 ptos.** por aplicar correctamente el efecto según el valor del *switch*; y **0.25 ptos.** por enviar correctamente el resultado al conversor digital-análogo.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2024)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (1.5 ptos.) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número 0,1101101b. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo; 3 bits de parte entera; y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante con la siguiente distribución: 1 bit de signo de significante; 5 bits de significante; 1 bit de signo de exponente; y 1 bit de exponente. ¿Cuál de estas representaciones presenta una menor pérdida de precisión para almacenar el número? Justifique su respuesta.

Nota: Durante la evaluación, se aclara que en punto flotante se usa la representación con bit explícito, no implícito.

Solución: Para este número, es recomendable hacer uso de la representación de **punto flotante**. Su representación normalizada equivale a $1,101101 \times 10^{-1}$, por lo que **con bit explícito** se almacena la siguiente secuencia: $0_{\text{Signo sig.}}11011_{\text{Significante}}1_{\text{Signo exp.}}1_{\text{Exp.}}$. De este resultado, se deduce que se pierden los dos bits menos significativos, equivalente a una pérdida de precisión de 2^{-7}. En cambio, en el formato de punto fijo, se almacena la siguiente secuencia: $0_{\text{Signo}}000_{\text{Entero}}1101_{\text{Fracción}}$. Se observa que en esta última representación se pierden los 3 bits menos significativos, por lo que su resultado es menos preciso al tener una pérdida de precisión de $2^{-5} + 2^{-7}$. Se otorgan **0.75 ptos.** por indicar la representación que implica menor pérdida de precisión; y **0.75 ptos.** por justificar indicando la cantidad de valor o bits perdidos en cada caso.

(b) (1.5 ptos.) Indique el valor que representa el float 0x7FC00010. Justifique su respuesta en base al estándar IEEE754.

Solución: Como el exponente desfasado del número es 255 (*i.e.* todos sus bits iguales a 1), que se reserva para ciertas representaciones, su valor puede ser $\pm\infty$ o `NaN` (*Not a Number*). Luego, su significante posee al menos dos bits distintos de cero, por lo que su valor equivale a `NaN`. Se otorgan **0.75 ptos.** por señalar correctamente el valor que representa el float; y **0.75 ptos.** por justificación en función de su exponente y significante.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar tanto el valor teórico (resultado de la operación) como el valor real (almacenado como float del estándar IEEE754) de la operación. Si son iguales, explicite que lo son. Los valores deben presentarse en base binaria con notación científica normalizada.

- (1.5 ptos.) $A + B$. $A =$ 0x40400000, $B =$ 0x3F800000.

Solución:

- $A =$0100000001000000 00000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10000000b $\rightarrow$ 128 - 127 = 1.
  - **Significante:** 10000000000000000000000b $\rightarrow$ 1,1b.
  - **Valor:** $(1,1 \times 10^1)$b = 11b = 3.
- $B =$ 00111111100000000000000000000000.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 01111111b $\rightarrow$ 127 - 127 = 0.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** $(1,0 \times 10^0)$b = 1b = 1.

De lo anterior, se deduce que la operación a realizar es 3 + 1. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,1 \times 10^1$. Luego, sumando los significantes, se tiene $A + B = (10,0 \times 10^1)$b = $(1,0 \times 10^{10})$. Finalmente, si se almacena como `float`, se tiene el valor 01000000100000000000000000000000b = 0x40800000 = 4. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de cada número; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

- (1.5 ptos.) $A \times B$. A = 0x41820000, B = 0x3F200000.

Solución:

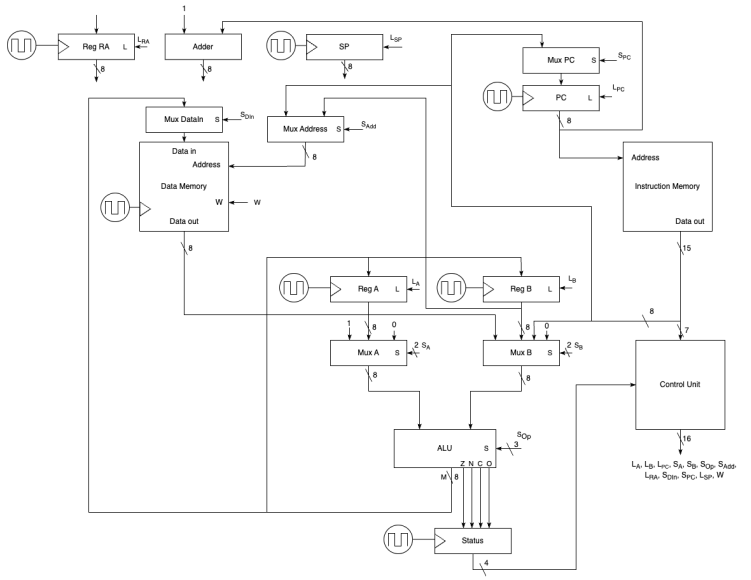
- A = 01000001100000100000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000011b → 131 - 127 = 4.
  - **Significante:** 00000100000000000000000b → 1,000001b.
  - **Valor:** $(1,000001 \times 10^{100})$b = 10000,01b = 16,25.
- B = 00111111001000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 01111110b → 126 - 127 = -1.
  - **Significante:** 01000000000000000000000b → 1,01b.
  - **Valor:** $1,01 \times 10^{-1} = 0,101$b = 0,625.

De lo anterior, se deduce que la operación a realizar es igual a $16,25 \times 0,625$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $-1 + 4 = 3 = 11$b. Luego, se multiplican los significantes: $1,01 \times 1,000001 = (101 \times 10^{-2}) \times (1000001 \times 10^{-6}) = (101 \times 1000001) \times 10^{-8} = 101000101 \times 10^{-8} = 1,01000101$. Finalmente, se tiene como resultado $1,01000101 \times 10^{11}$b = 1010,00101 = 10,15625, que en formato float se almacena como 01000001001000101000000000000000b = 0x41228000. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de cada número; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 2: Computador básico con subrutinas (6 ptos.)

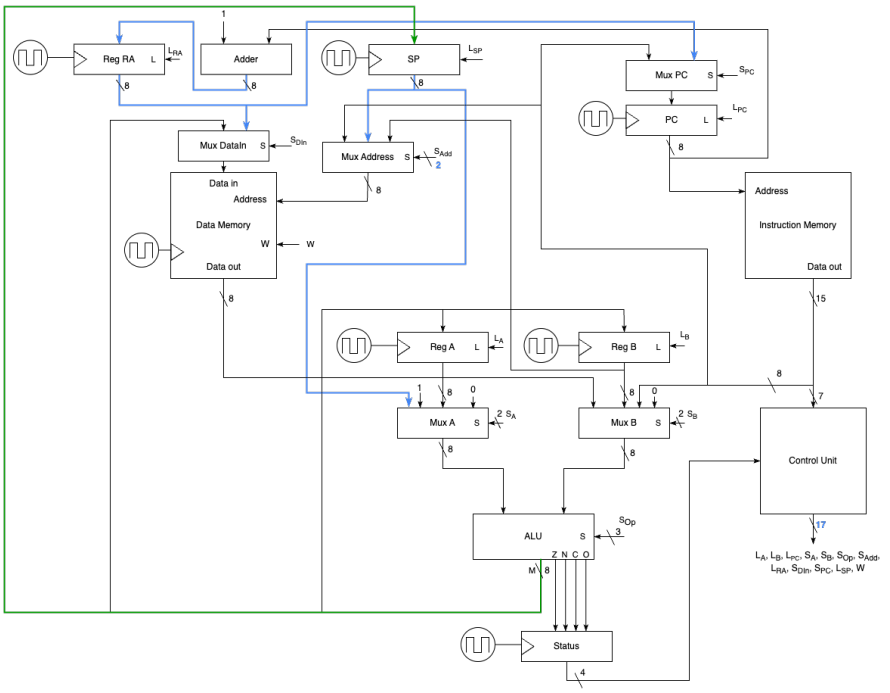
En arquitecturas RISC, las direcciones de retorno de una subrutina se almacenan en un **registro de enlace** (`RA`), a diferencia del computador básico donde se guardan en el *stack*. Asimismo, estas arquitecturas no poseen `PUSH/POP`, sino que acceden al *stack* directamente con el *stack pointer* (`SP`). Deberá contestar las preguntas de esta sección a partir de este contexto y el siguiente diagrama:



(a) **(5 ptos.)** A partir del diagrama de base adjunto, y asumiendo que se eliminan las instrucciones `CALL`, `RET`, `PUSH` y `POP`, realice las modificaciones de *hardware* necesarias e indique la combinación de señales completa para ejecutar las siguientes instrucciones en un ciclo:

- **(0.5 ptos.)** `MOV A,(SP)`. Guarda en `A` el valor `Mem[SP]`.
- **(0.5 ptos.)** `MOV (SP), A`. Guarda en `Mem[SP]` el valor `A`.
- **(1 pto.)** `ADD SP,Lit`. Guarda en `SP` el valor `SP + Lit`. `Lit` puede ser negativo.
- **(1 pto.)** `MOV (SP), RA`. Guarda en `Mem[SP]` el valor `RA`.
- **(1 pto.)** `JAL RA, label`. Guarda `PC+1` en `RA` y salta a la dirección asociada a `label`.
- **(1 pto.)** `JALR RA`. Salta a la dirección almacenada en `RA`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- La conexión entre `SP` y `Mux Address` permite el uso de `SP` como dirección de memoria con la señal S_{Add}, la que ahora debe ser de 2 bits.
- La conexión entre `SP` y `Mux A` permite su uso como primer operando en la `ALU`.
- La conexión entre la salida de la `ALU` y `SP` permite cargar en este último los resultados de las operaciones con la señal L_{SP}.
- La conexión entre `RA` y `Mux DataIn` permite la escritura de `RA` en memoria.
- La conexión entre `RA` y `PC+1` permite escribir este valor en `RA` con la señal L_{RA}.
- La conexión entre `RA` y `Mux PC` permite realizar saltos hacia direcciones almacenadas en `RA`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA

Por cada instrucción, se otorga **la mitad** del puntaje por *hardware* y **la otra mitad** por la combinación de señales. En cada criterio, se descuenta la mitad del puntaje en caso de existir, como máximo, un error de implementación o valor en la combinación de señales. En otro caso, no se otorga puntaje.

(b) **(1 pto.)** Modifique el siguiente fragmento de código para que haga uso exclusivo de las instrucciones anteriores. Debe mantener su funcionamiento original y terminar sin errores.

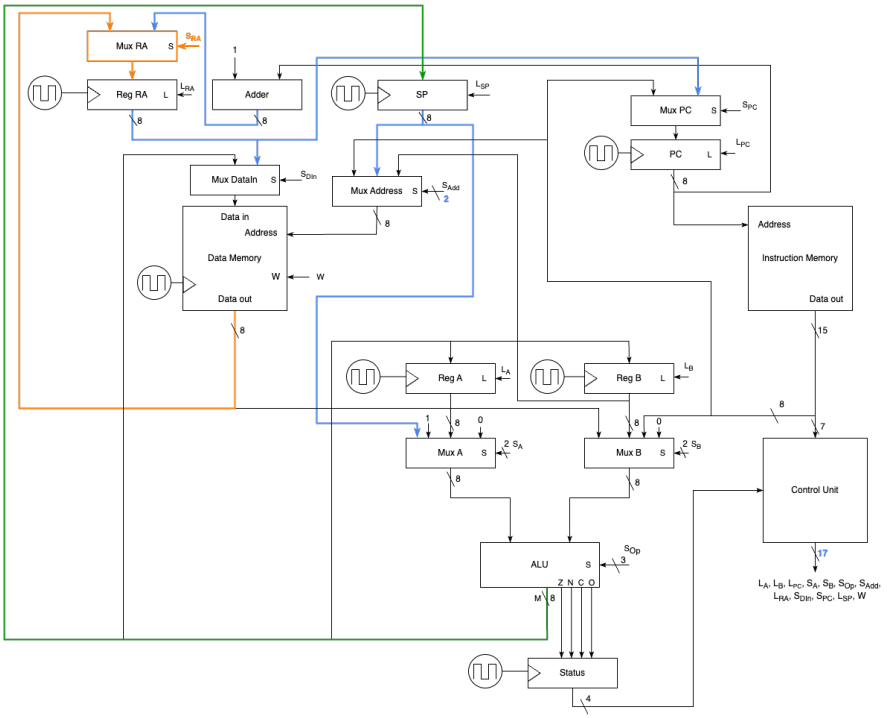
```
MOV A,1
PUSH A
CALL func
POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  CALL func
  end_func:
    RET
end:
```

Solución: Es importante señalar que, con las instrucciones entregadas, **no es posible** entregar un fragmento de código que mantenga el funcionamiento original **sin errores**. Esto se debe al hecho de que no existe una instrucción que permita recuperar una dirección de retorno respaldada en memoria (por ejemplo, `MOV RA, (SP)`. Por este motivo, se consideran correctas respuestas que solo caigan en ese caso. A continuación, una solución posible.

```
MOV A,1
MOV (SP),A        ; Reemplazo por PUSH A
ADD SP,-1         ; Reemplazo por PUSH A
JAL RA,func       ; Reemplazo por CALL func
ADD SP,1          ; Reemplazo por POP A
MOV A,(SP)        ; Reemplazo por POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  MOV (SP),RA     ; Respaldo de RA para evitar error en llamado recursivo
  ADD SP,-1       ; Respaldo de RA para evitar error en llamado recursivo
  JAL RA,func     ; Reemplazo por CALL func
  ADD SP,1        ; De todas formas, se restituye el valor de SP
  MOV RA,(SP)     ; Si existiera, podríamos recuperar el valor respaldado de RA
  end_func:
    JALR RA       ; Reemplazo por RET
end:
```

Se otorgan **0.25 ptos.** por reemplazar correctamente la instrucción `PUSH A`; **0.25 ptos.** por reemplazar correctamente la instrucción `CALL label`; **0.25 ptos.** por reemplazar correctamente la instrucción `POP A`; y **0.25 ptos.** por reemplazar correctamente la instrucción `RET`. Adicionalmente, si la respuesta alude a la falta de una instrucción para recuperar la dirección de retorno; o bien se usa de forma hipotética una instrucción que permita resolver el problema, **se otorgarán 0.5 ptos. de bonus al total**.

A modo de completitud, se incluye tanto el diagrama como la tabla de señales de la arquitectura que habilita la ejecución de la instrucción `MOV RA,(SP)`.



Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{RA}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-	-
`MOV RA, (SP)`	0	0	0	0	1	0	-	-	-	SP	-	-	DOUT
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT	PC+1
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA	-

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Si un computador ya tiene dispositivos *memory mapped* a través de un *address decoder*, ¿se necesita *hardware* adicional para habilitar la comunicación vía *polling*? Justifique.

> **Solución: No**, no es necesario ya que en la comunicación vía *polling* es la CPU la que se encarga de interactuar con los dispositivos a través de la revisión de sus estados, lo que ya puede realizar con el acceso directo a las direcciones de memoria asociadas a ellos. Se otorgan **0.5 ptos.** por responder correctamente; y **0.5 ptos.** por justificación.

(b) (1 pto.) ¿Cuál es la función del vector de interrupciones? ¿Cuál es su contenido?

> **Solución:** La función del vector de interrupciones es permitir la ejecución correcta de la ISR (*Interruption Service Routine*) a partir del identificador del dispositivo a ser atendido, lo que es gestionado por el controlador de interrupciones. Este vector contiene los **punteros** de las ISR, por lo que si se desea atender al dispositivo `i`, se ejecuta el llamado de la subrutina a partir de su dirección igual a `vector_interrupciones[i]`. Se otorgan **0.5 ptos.** por señalar correctamente la función del vector de interrupciones; y **0.5 ptos.** por indicar de forma correcta su contenido.

(c) (4 ptos.) Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	**Nombre**	**Descripción**
0x00	`coffee_grinder_status`	Registro estado molinillo
0x04	`coffee_grinder_command`	Registro comando molinillo
0x08	`coffee_grinder_grind_size`	Config. grosor molienda
0x0C	`coffee_grinder_weight`	Config. gramos a moler
0x10	`kettle_status`	Registro estado hervidor
0x14	`kettle_command`	Registro comando hervidor
0x18	`kettle_temperature`	Config. temperatura hervidor (ºC)

Nombre	**Descripción**	**Valor**
`coffee_grinder_status`	Apagado	0
`coffee_grinder_status`	Encendido	1
`coffee_grinder_status`	Moliendo	2
`kettle_status`	Apagado	0
`kettle_status`	Encendido	1
`kettle_status`	Hirviendo	2
`coffee_grinder_command`	Encender	1
`coffee_grinder_command`	Apagar	255
`coffee_grinder_command`	Moler	128
`kettle_command`	Encender	1
`kettle_command`	Apagar	255
`kettle_command`	Hervir	128
`coffee_grinder_grind_size`	Fino	1
`coffee_grinder_grind_size`	Medio	2
`coffee_grinder_grind_size`	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
step_two:
    sw s2, 8(t0)
    sw s5, 12(t0)
    sw s3, 4(t0)
    jal ra, step_five
# Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)
step_four:
    sw s6, 24(t0)
    sw s3, 20(t0)
    jal ra, step_six
    beq zero, zero, end
step_five:
    while_step_five:
        lw t1, 0(t0)
        beq t1, s2, while_step_five
    sw s4, 4(t0)
    jalr zero, 0(ra) # Retorno
step_six:
    while_step_six:
        lw t1, 16(t0)
        beq t1, s2, while_step_six
    sw s4, 20(t0)
    jalr zero, 0(ra)
end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.7 ptos.)**: Se revisa el estado del molinillo. Si está moliendo, se ejecuta `step_five` y se salta a `step_three`. Si está encendida, se salta a `step_two`. En otro caso, se enciende antes de continuar con `step_two`.
- `step_two` **(0.7 ptos.)**: Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta `step_five` antes de seguir a `step_three`.
- `step_three` **(0.7 ptos.)**: Se revisa el estado del hervidor. Si está hirviendo, se ejecuta `step_six` y se termina el programa. Si está encendida, se salta a `step_four`. En otro caso, se enciende antes de continuar con `step_four`.
- `step_four` **(0.7 ptos.)**: Se configura el hervidor con una temperatura de 92ºC. Luego, empieza a hervir y se ejecuta `step_six` antes de terminar el programa.
- `step_five` **(0.6 ptos.)**: Se revisa el estado del molinillo hasta que deje de moler.
- `step_six` **(0.6 ptos.)**: Se revisa el estado del hervidor hasta que deje de hervir.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Compare la representación de punto fijo con la representación de punto flotante en términos de rango y precisión. ¿Cuál es más conveniente en cada caso y por qué?

Solución: La representación de punto fijo es más conveniente si lo que se busca es **precisión**, dado que sus valores se distribuyen de manera uniforme en su rango gracias a la porción de bits de parte fraccional. No obstante, tiene un rango muy acotado que depende de los bits restantes para representar la parte entera del número. La representación de punto flotante, en cambio, es más conveniente si lo que se busca es un **rango mayor**, ya que el uso de bits para representar el exponente permite "flotar" el punto y representar valores mucho más grandes o pequeños. Hay que considerar, de todas formas, que mientras más grande o pequeño sea el valor a representar, su precisión disminuye significativamente al depender de los bits restantes para el significante. Se otorga **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto fijo, y **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto flotante.

(b) (1 pto.) Suponga que imprime en la consola de su computador un valor que se encuentra almacenado en formato float del estándar IEEE754. Al imprimir, en la consola ve que aparece el valor "`NaN`". ¿Es posible, a partir de este resultado, obtener la secuencia de 32 bits exacta que representa este valor? Justifique.

Solución: No es posible. La representación del valor `NaN` del formato float tiene las siguientes características:

- Su exponente es igual a 11111111.
- **Al menos uno de sus bits de significante es distinto de cero.**

El último punto es relevante. Dado que cualquiera de los bits del significante es distinto de cero, al leer en consola el valor "`NaN`" no tenemos cómo saber cuál de los 23 bits es igual a 1. Se otorgan **0.5 ptos.** por responder correctamente, y **0.5 ptos.** por justificación correcta. Si solo responde que no es posible sin ninguna justificación, no se otorga puntaje.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (1.5 ptos.) $A + B$. $A =$ 0x40400000, $B =$ 0x80000000.

Solución:

- $A = 0$ 10000000 10000000000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10000000b $\rightarrow$ 128 - 127 = 1.
  - **Significante:** 10000000000000000000000b $\rightarrow$ 1,1b.
  - **Valor:** $(1,1 \times 10^{1})$b = 11b = 3.
- $B = 1$ 00000000 00000000000000000000000b.
  - **Signo:** 1b $\rightarrow$ negativo.
  - **Exponente:** 00000000b $\rightarrow$ **reservado para el cero**.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** -0 (valor reservado).

De lo anterior, se deduce que la operación a realizar es $3 + (-0)$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a 3** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por 0, manteniendo el valor final de A.

- (1.5 ptos.) $A \times B$. A = 0x40000000, B = 0xC0A00000.

Solución:

- A = 0 10000000 00000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^1)$b = 10b = 2.
- B = 1 10000001 01000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 10000001b → 129 - 127 = 2.
  - **Significante:** 01000000000000000000000b → 1,01b.
  - **Valor:** $-(1,01 \times 10^2)$b = $-$101b = -5.

De lo anterior, se deduce que la operación a realizar es 2×-5. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1 + 2 = 3 = 11$b. Luego, se multiplican los significantes: $1,0 \times -1,01 = -1,01$. Finalmente, se tiene como resultado $-1,01 \times 10^{11}$b = $-$1010b = $-$10, que en formato float se almacena como 11000001001000000000000000000000b = 0xC1200000. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 2: Computador básico con subrutinas (6 ptos.)

En los ítems **(a)** y **(b)**, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo**.

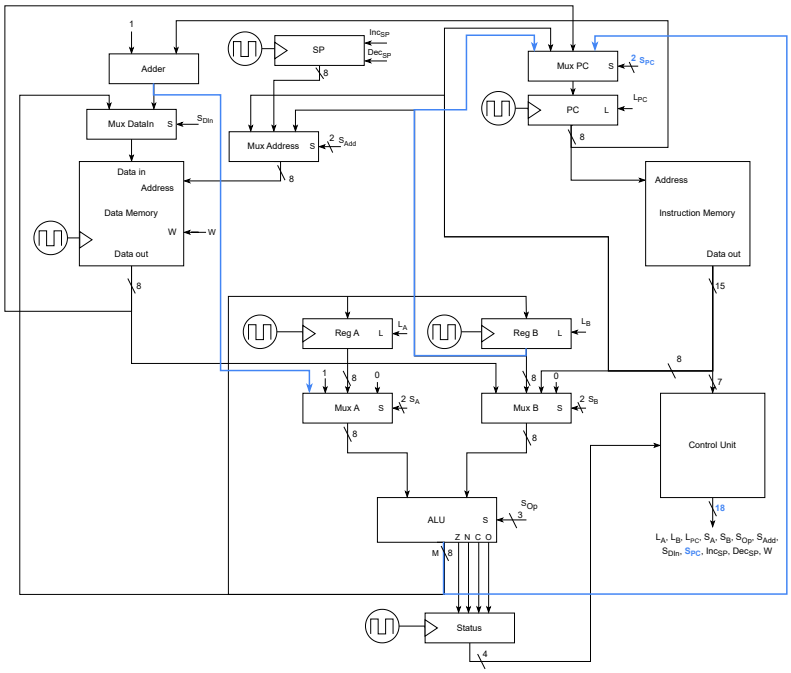
(a) (0.75 ptos.) `CALL A,B`. Almacena en el registro `A` la dirección de retorno `PC+1` y salta a la dirección de memoria almacenada en el registro `B`.

(b) (0.75 ptos.) `CALL B,Dir`. Almacena en el registro `B` la dirección de retorno `PC+1` y salta a la dirección de memoria equivalente al literal `Dir`.

(c) (0.75 ptos.) `RET A`. Salta a la dirección de retorno almacenada en el registro `A`.

(d) (0.75 ptos.) `RET B`. Salta a la dirección de retorno almacenada en el registro `B`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se conecta la salida del *Adder* que computa `PC+1` con la entrada restante del componente `Mux A`, de forma que se pueda tener este valor como salida de la ALU para su almacenamiento en los registros `A` o `B`.
- Se conecta la salida de la ALU con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor del registro `A` o del registro `B` como direcciones de memoria a saltar.
- Se conecta la salida del registro `B` con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor de este como dirección de memoria a saltar mientras que al mismo tiempo se compute `PC+1` desde la ALU.
- Finalmente, incrementa a 2 bits la señal S_{PC} para poder soportar las dos nuevas entradas del multiplexor `Mux PC`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
CALL A,B	1	0	1	0	0	0	PC+1	ZERO	ADD	-	-	B
CALL B,Dir	0	1	1	0	0	0	PC+1	ZERO	ADD	-	-	LIT
RET A	0	0	1	0	0	0	A	ZERO	ADD	-	-	ALU
RET B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Por cada instrucción, se otorgan:

- **0.375 ptos.** por *hardware* correctamente modificado; **0.25 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.375 ptos.** por entregar una combinación de señales correcta; **0.25 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

(e) (3 ptos.) A continuación, se adjuntan dos fragmentos de código **(1)** y **(2)** escritos en el Assembly del computador básico, con la diferencia de que **(2)** incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros `A` y `B` al finalizar la ejecución del código. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
  var1 12
  var2 4
CODE:
  MOV A,5     ; Dir. memoria 0x00
  MOV B,3     ; Dir. memoria 0x01
  RET         ; Dir. memoria 0x02-0x03
  case_1:
    ADD A,B   ; Dir. memoria 0x04
    PUSH A    ; Dir. memoria 0x05
    MOV A,B   ; Dir. memoria 0x06
    SHL A,A   ; Dir. memoria 0x07
    MOV B,A   ; Dir. memoria 0x08
    POP A     ; Dir. memoria 0x09-0x0A
    JMP end   ; Dir. memoria 0x0B
  case_2:
    SUB A,B   ; Dir. memoria 0x0C
    PUSH A    ; Dir. memoria 0x0D
    MOV A,B   ; Dir. memoria 0x0E
    SHR A,A   ; Dir. memoria 0x0F
    MOV B,A   ; Dir. memoria 0x10
    POP A     ; Dir. memoria 0x11-0x12
  end:
```

(2)

```
DATA:
  n    2
  res  0
CODE:
  MOV A,(n)       ; Dir. memoria 0x00
  CALL B,fun      ; Dir. memoria 0x01
  JMP end         ; Dir. memoria 0x02
  fun:
    CMP A,0       ; Dir. memoria 0x03
    JLE fun_end   ; Dir. memoria 0x04
    PUSH B        ; Dir. memoria 0x05
    MOV B,A       ; Dir. memoria 0x06
    ADD A,(res)   ; Dir. memoria 0x07
    MOV (res),A   ; Dir. memoria 0x08
    MOV A,B       ; Dir. memoria 0x09
    SUB A,1       ; Dir. memoria 0x0A
    CALL B,fun    ; Dir. memoria 0x0B
    POP B         ; Dir. memoria 0x0C-0x0D
    fun_end:
      RET B       ; Dir. memoria 0x0E

  end:
    MOV A,(res)   ; Dir. memoria 0x0F
```

Solución: El fragmento de código **(1)** accede, por *overflow* del registro `SP`, a la dirección de memoria 0 (equivalente a `var1`) para cargar su valor en el registro `PC`, por lo que la ejecución de `RET` genera el salto al *label* `case_2`. El fragmento de código **(2)**, por otra parte, computa de forma recursiva la suma de la variable `n` con todos sus antecesores hasta llegar a cero, almacenando el resultado final en `res`. A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
  var1 12     ; MEM[0] = 12
  var2 4      ; MEM[1] = 4
CODE:
  MOV A,5     ; A = 5
  MOV B,3     ; B = 3
  RET         ; SP++ => 255 + 1 = 0, PC = MEM[0] = 12
  case_1:
    ADD A,B   ; NO SE EJECUTA
    PUSH A    ; NO SE EJECUTA
    MOV A,B   ; NO SE EJECUTA
    SHL A,A   ; NO SE EJECUTA
    MOV B,A   ; NO SE EJECUTA
    POP A     ; NO SE EJECUTA
    JMP end   ; NO SE EJECUTA
  case_2:
    SUB A,B   ; A = 2
    PUSH A    ; MEM[SP] = A => MEM[0] = 2, SP-- => 0 - 1 = 255
    MOV A,B   ; A = 3
    SHR A,A   ; A = 1
    MOV B,A   ; B = 1
    POP A     ; SP++ => 255 + 1 = 0, A = MEM[0] = 2
  end: ; A = 2, B = 1
```

(2)

```
DATA:
  n    2          ; MEM[0] = 2
  res  0          ; MEM[1] = 0
CODE:
  MOV A,(n)       ; A = n = 2
  CALL B,fun      ; B = 2, fun(n)
  JMP end
  fun:
    CMP A,0
    JLE fun_end   ; Termina cuando A = 0
    PUSH B        ; Respalda B en stack (retorno)
    MOV B,A       ; B = A = n
    ADD A,(res)   ; A += res
    MOV (res),A   ; res = res + n
    MOV A,B       ; A = n
    SUB A,1       ; n--
    CALL B,fun    ; fun(n-1)
    POP B         ; Recupera B (retorno)
    fun_end:
      RET B       ; Retorna de la forma esperada
  end:
    ; A = res = fun(2) = 2+fun(1) = 2+1+fun(0) = 2+1+0 = 3
    MOV A,(res) ; B = Primer retorno computado = 2
```

Para cada fragmento de código, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Señale una ventaja y una desventaja de mapear los dispositivos I/O de un computador en memoria.

Solución: Una ventaja es que no implica modificaciones en la ISA, lo que permite comunicarse directamente con los dispositivos a través de instrucciones de lectura y escritura de memoria. Una desventaja, en cambio, es el uso significativo de direcciones de memoria a reservar, que en memorias más pequeñas puede implicar problemas de ejecución en los programas (*memory barrier*). Se otorgan **0.5 ptos.** por describir correctamente una ventaja y **0.5 ptos.** por describir correctamente una desventaja.

(b) (1 pto.) Explique el uso e importancia de las ISRs ejecutadas por la CPU en el contexto de comunicación vía interrupciones.

Solución: Las ISRs o *Interrupt Service Routines* son subrutinas asociadas al manejo que requieren los dispositivos por parte de la CPU una vez que son atendidas por la CPU. Son importantes ya que, a través de su ejecución, se lleva a cabo la comunicación entre la CPU y los dispositivos. Se otorgan **0.5 ptos.** por describir correctamente en qué consiste una ISR y **0.5 ptos.** por destacar su importancia.

(c) (4 ptos.) Suponga que decide automatizar la disponibilidad de un baño mediante sensores y un emisor Bluetooth conectados a un computador con arquitectura RISC-V. Para la comunicación con los sensores y el emisor, sus registros de 32 bits están mapeados en memoria desde la dirección `0x600DD0D0`:

Offset	**Nombre**	**Descripción**
0x00	`door_sensor_status`	Registro estado sensor de puerta
0x04	`light_sensor_status`	Registro estado sensor de luz
0x08	`notifier_command`	Registro comando emisor
0x0C	`notifier_status`	Registro estado emisor

Nombre	**Descripción**	**Valor**
`door_sensor_status`	Puerta cerrada	0
`door_sensor_status`	Puerta abierta	1
`light_sensor_status`	Luz apagada	0
`light_sensor_status`	Luz encendida	1
`notifier_status`	Inactivo	0
`notifier_status`	Activo	1
`notifier_status`	Enviando	2
`notifier_command`	Encender emisor	1
`notifier_command`	Notificar `OPEN`	2
`notifier_command`	Notificar `CLOSED`	3
`notifier_command`	Apagar emisor	4

El sistema debe enviar una notificación Bluetooth según la disponibilidad del baño: `OPEN` si es que está disponible, y `CLOSED` si es que está ocupado según los estados de los sensores de puerta (`door_sensor_status`) y luz (`light_sensor_status`). Para ello, se desarrolla el siguiente programa:

```
notifier_automation:
    # Carga zero + 0x600DD0D0 en t0
    addi t0, zero, 0x600DD0D0
    addi s0, zero, 0
    addi s1, zero, 1
    addi s2, zero, 2
    addi s3, zero, 3
    addi s4, zero, 4

step_one:
    lw t3, 12(t0) # Carga Mem[t0 + 12] en t3
    # Salta a step_two si t3 != s0
    bne t3, s0, step_two
    sw s1, 8(t0) # Carga s1 en Mem[t0 + 8]

step_two:
    lw t1, 0(t0)
    lw t2, 4(t0)
    bne t1, s1, step_three
    bne t2, s0, step_three
    # Salto incondicional a step_four
    jal zero, step_four
```

```
# => Continuación

step_three:
    sw s3, 8(t0)

    while_step_three:
        lw t3, 12(t0)
        # Salta a while_step_three si t3 == s2
        beq t3, s2, while_step_three

    sw s4, 8(t0)
    jal zero, step_one

step_four:
    sw s2, 8(t0)

    while_step_four:
        lw t3, 12(t0)
        beq t3, s2, while_step_four

    sw s4, 8(t0)
    jal zero, step_one
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(1 pto.)**: Se revisa el estado del emisor. Si el emisor está inactivo, se enciende y continúa a `step_two`. En otro caso, salta directamente a `step_two`.
- `step_two` **(1 pto.)**: Se revisa el estado del sensor de puerta y el sensor de luz del baño. Si la puerta está cerrada o la luz está encendida, salta a `step_three`. En otro caso, salta directamente a `step_four`.
- `step_three` **(1 pto.)**: A través del emisor, se envía la notificación `CLOSED`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.
- `step_four` **(1 pto.)**: A través del emisor, se envía la notificación `OPEN`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.

Para cada paso, se descuenta un 25 % del puntaje si existe como máximo un error o detalle faltante; un 50 % si existen como máximo dos errores; y un 100 % en otro caso.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 2 - Sección 1

Pauta de evaluación

Pregunta 2: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Indique cuántos bits de precisión posee la representación de un número float del estándar IEEE754 y explique por qué.

> **Solución:** La representación de floats del estándar IEEE754 cuenta con **24 bits de precisión**. Esto se debe a la inclusión del bit implícito, el que otorga un bit de precisión adicional a los 23 bits del significante. Se otorga **1 pto.** por señalar la cantidad correcta de bits de precisión y **1 pto.** por justificación.

(b) (4 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (2 ptos.) $A + B$. $A =$ 0xBF800000, $B =$ 0x7F800008.

> **Solución:**
>
> - $A =$ 1 01111111 00000000000000000000000b.
> - **Signo:** 1b → negativo.
> - **Exponente:** 01111111b → 127 - 127 = 0.
> - **Significante:** 00000000000000000000000b → 1,0b.
> - **Valor:** $-(1,0 \times 10^0)$b $= -1$b $= -1$.
> - $B =$ 0 11111111 00000000000000000001000b.
> - **Signo:** 0b → positivo.
> - **Exponente:** 11111111b → **reservado para infinito y `NaN`**.

- **Significante:** 00000000000000000001000b → distinto de $1,0$.
- **Valor:** `NaN` (valor reservado).

De lo anterior, se deduce que la operación a realizar es $-1 + \texttt{NaN}$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a `NaN`** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float. En este caso, en estricto rigor la representación en formato float del resultado podría o no ser igual a B ya que depende de la implementación, por lo que se considera correcto si deducen que el resultado es `NaN` independiente de la representación que utilicen.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por `NaN`, manteniendo el valor final de B.

- (2 ptos.) $A \times B$. $A =$ 0xC0000000, $B =$ 0x42000000.

Solución:

- $A =$ 1 10000000 00000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $-(1,0 \times 10^1)$b $= -10$b $= -2$.
- $B =$ 0 10000100 00000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000100b → 132 - 127 = 5.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^5)$b $= 100000$b $= 32$.

De lo anterior, se deduce que la operación a realizar es -2×32. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1+5 = 6 = 110$b. Luego, se multiplican los significantes: $-1,0 \times 1,0 = -1,0$. Finalmente, se tiene como resultado $-1,0 \times 10^{110}$b $= -1000000$b $= -64$, que en formato float se almacena como 11000010100000000000000000000000b = 0xC2800000. **No existe pérdida de precisión**.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1.5 ptos.) Explique el significado y uso del identificador que envía el controlador de interrupciones luego de recibir la señal INTA por parte de la CPU.

Solución: Al recibir la señal INTA, la CPU espera obtener el identificador asociado al **dispositivo cuya interrupción será atendida**. Este será enviado por el controlador de interrupciones. Luego, la CPU hace uso de este valor como índice del vector de interrupciones para obtener el puntero de la ISR del dispositivo atendido, permitiendo así la ejecución de su subrutina. Se otorgan **0.75 ptos.** por indicar la asociación entre el identificador y los dispositivos I/O; y **0.75 ptos.** por indicar su uso en la ejecución de la ISR deseada.

(b) (1.5 ptos.) Indique qué diferencia existe entre una interrupción gatillada por un dispositivo I/O y una gatillada por *software*.

Solución: La **única** diferencia es la señalada en el enunciado: una se gatilla desde un dispositivo I/O, mientras que la otra se gatilla por *software* (ya sea por una excepción o por un *trap*). En ambos casos, la interrupción se lleva a cabo a través de la ejecución de una ISR cuyo puntero se almacena en el vector de interrupciones, y ambas cuentan con un identificador que envía el controlador de interrupciones al recibir la señal INTA por parte de la CPU. Se otorgan **0.75 ptos.** por señalar la fuente de la interrupción como diferencia, y **0.75 ptos.** por explicar por qué no se diferencian en nada más.

(c) (3 ptos.) Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. A continuación, una representación gráfica de su implementación:



Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	Nombre	Descripción
0x00	`coffee_grinder_status`	Registro estado molinillo
0x04	`coffee_grinder_command`	Registro comando molinillo
0x08	`coffee_grinder_grind_size`	Config. grosor molienda
0x0C	`coffee_grinder_weight`	Config. gramos a moler
0x10	`kettle_status`	Registro estado hervidor
0x14	`kettle_command`	Registro comando hervidor
0x18	`kettle_temperature`	Config. temperatura hervidor (ºC)

Nombre	Descripción	Valor
`coffee_grinder_status`	Apagado	0
`coffee_grinder_status`	Encendido	1
`coffee_grinder_status`	Moliendo	2
`kettle_status`	Apagado	0
`kettle_status`	Encendido	1
`kettle_status`	Hirviendo	2
`coffee_grinder_command`	Encender	1
`coffee_grinder_command`	Apagar	255
`coffee_grinder_command`	Moler	128
`kettle_command`	Encender	1
`kettle_command`	Apagar	255
`kettle_command`	Hervir	128
`coffee_grinder_grind_size`	Fino	1
`coffee_grinder_grind_size`	Medio	2
`coffee_grinder_grind_size`	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
step_two:
    sw s2, 8(t0)
    sw s5, 12(t0)
    sw s3, 4(t0)
    jal ra, step_five
# Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)
step_four:
    sw s6, 24(t0)
    sw s3, 20(t0)
    jal ra, step_six
    beq zero, zero, end
step_five:
    while_step_five:
        lw t1, 0(t0)
        beq t1, s2, while_step_five
    sw s4, 4(t0)
    jalr zero, 0(ra) # Retorno
step_six:
    while_step_six:
        lw t1, 16(t0)
        beq t1, s2, while_step_six
    sw s4, 20(t0)
    jalr zero, 0(ra)
end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.5 ptos.)**: Se revisa el estado del molinillo. Si está moliendo, se ejecuta `step_five` y se salta a `step_three`. Si está encendida, se salta a `step_two`. En otro caso, se enciende antes de continuar con `step_two`.
- `step_two` **(0.5 ptos.)**: Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta `step_five` antes de seguir a `step_three`.
- `step_three` **(0.5 ptos.)**: Se revisa el estado del hervidor. Si está hirviendo, se ejecuta `step_six` y se termina el programa. Si está encendida, se salta a `step_four`. En otro caso, se enciende antes de continuar con `step_four`.
- `step_four` **(0.5 ptos.)**: Se configura el hervidor con una temperatura de 92ºC. Luego, empieza a hervir y se ejecuta `step_six` antes de terminar el programa.
- `step_five` **(0.5 ptos.)**: Se revisa el estado del molinillo hasta que deje de moler.
- `step_six` **(0.5 ptos.)**: Se revisa el estado del hervidor hasta que deje de hervir.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 2 - Sección 2

Pauta de evaluación

Pregunta 2: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Indique cuántos exponentes distintos se pueden representar en un número float del estándar IEEE754 y explique por qué.

Solución: Como los exponentes 00000000b y 11111111b se reservan para las representaciones de 0 y $\pm\infty$/`NaN` respectivamente, el rango de exponentes de los floats bajo el estándar IEEE754 corresponde a $[-126, 127]$, es decir, se tienen 254 exponentes distintos (256 combinaciones distintas - 2 reservadas). Se otorga **1 pto.** por indicar correctamente la cantidad de exponentes distintos, y **1 pto.** por justificación.

(b) (4 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (2 ptos.) $A + B$. A = 0x58800000, B = 0x3F800000.

Solución:

- A = 0 10110001 0000000 00000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10110001b $\rightarrow$ 177 - 127 = 50.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** $(1,0 \times 10^{50})$b = 10000..,000b = 2^{50}.
- B = 0 01111111 00000000000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.

- **Exponente:** 01111111b → 127 - 127 = 0.
- **Significante:** 00000000000000000000000b → 1,0b.
- **Valor:** $(1,0 \times 10^0)$b = 1b = 1.

De lo anterior, se deduce que la operación a realizar es $2^{50} + 1$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,0000..,001 \times 10^{50}$. Luego, sumando los significantes, se tiene $A + B = (1,0000..,001 \times 10^{50})$b. Finalmente, si se almacena como `float`, se tiene el valor 01011000100000000000000000000000b = 0x58800000 = 2^{50}. **Se pierden los 27 bits menos significantes que aportaban la magnitud de 1 y, por ende, se pierde precisión**.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

- (2 ptos.) $A \times B$. A = 0xBF800000, B = 0x7F800000.

Solución:

- A = 1 01111111 00000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 01111111b → 127 - 127 = 0.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $-(1,0 \times 10^0)$b = −1b = −1.
- B = 01111111100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 11111111b → **reservado para infinito y `NaN`**
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** ∞.

De lo anterior, se deduce que la operación a realizar es igual a $-1 \times \infty$. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a** $-\infty$, cuya representación en float es igual a 11111111100000000000000000000000b = 0xFF800000. Por lo tanto, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

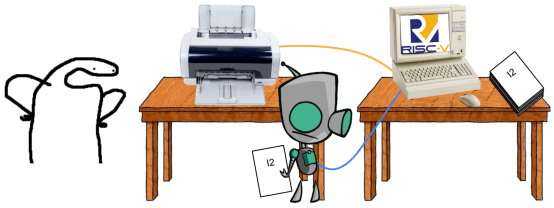
(a) (1.5 ptos.) ¿Qué pasaría, en el contexto de la comunicación con dispositivos I/O a través de interrupciones, si se eliminara la *flag* IF? Justifique su respuesta.

> **Solución:** La *Interruption Flag* le indica a la CPU si puede o no atender interrupciones. Si se elimina, la CPU no contaría con un mecanismo para determinar si debe detener su ejecución y atender una interrupción, impidiendo así la comunicación vía interrupciones. Otro posible escenario sería que se atiendan interrupciones aunque ya haya una en ejecución. Esto implicaría que no haya control en la ejecución anidada de ISRs y que se generen retrasos en su término (o incluso *stack overflow*). Se otorgan **0.75 ptos.** por señalar un posible escenario al eliminar la *flag*, y **0.75 ptos.** por justificación.

(b) (1.5 ptos.) ¿Cuál es la función del vector de interrupciones? ¿Cuál es su contenido?

> **Solución:** La función del vector de interrupciones es permitir la ejecución correcta de la ISR (*Interruption Service Routine*) a partir del identificador del dispositivo a ser atendido, lo que es gestionado por el controlador de interrupciones. Este vector contiene los **punteros** de las ISR, por lo que si se desea atender al dispositivo `i`, se ejecuta el llamado de la subrutina a partir de su dirección igual a `vector_interrupciones[i]`. Se otorgan **0.75 ptos.** por señalar correctamente la función del vector de interrupciones; y **0.75 ptos.** por indicar de forma correcta su contenido.

(c) (3 ptos.) Suponga que es el/la docente del curso y debe imprimir la I2. No obstante, la redacta a última hora y no alcanza a usar los equipos del DCC, por lo que decide imprimir en su hogar con un sistema innovador: una impresora y un robot conectados a un computador con ISA RISC-V. A continuación, una representación gráfica de su implementación:



Para llevar a cabo la comunicación, los registros de 32 bits de los dispositivos se mapean en memoria desde la dirección 0x00FF10AD:

Offset	Nombre	Descripción
0x00	`printer_status`	Registro de estado impresora
0x04	`printer_command`	Registro de comando impresora
0x08	`printer_copies`	Configuración de cantidad de copias impresora
0x0C	`printer_color`	Configuración de color impresora
0x10	`robot_status`	Registro de estado robot
0x14	`robot_command`	Registro de comando robot

Nombre	Descripción	Valor
`printer_status`	Apagado	0
`printer_status`	Encendido	1
`printer_status`	Imprimiendo	2
`robot_status`	Apagado	0
`robot_status`	Encendido	1
`robot_status`	Trabajando	2
`printer_command`	Encender	1
`printer_command`	Apagar	255
`printer_command`	Imprimir	128
`printer_color`	Blanco y negro	1
`printer_color`	Color	2
`robot_command`	Encender	1
`robot_command`	Rellenar papeles impresora	2
`robot_command`	Rellenar tinta impresora	3
`robot_command`	Apilar impresiones*	4
`robot_command`	Apagar	255

* Esta acción hace que el robot pase a estado “Trabajando”, el resto son instantáneas y no cambian su estado.

Para automatizar la impresión, desarrolla el siguiente programa:

```
printing_system:
# Carga 0 + 0x00FF10AD en t0
addi t0, zero, 0x00FF10AD
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 3
addi s4, zero, 4
addi s5, zero, 128
addi s6, zero, 255
addi s7, zero, 60

step_one:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_one si t1 != zero
    bne t1, zero, continue_step_one
    sw s1, 20(t0) # Carga s1 en Mem[t0 + 20]
    continue_step_one:
        lw t1, 16(t0)
        # Salta a continue_step_one si t1 == s2
        beq t1, s2, continue_step_one

step_two:
    lw t1, 0(t0)
    bne t1, zero, continue_step_two
    sw s1, 4(t0) # ---> Continúa a la derecha
```

```
# --> Continúa aquí
    continue_step_two:
        lw t1, 0(t0)
        beq t1, s2, continue_step_two

step_three:
    sw s7, 8(t0)
    sw s3, 20(t0)
    sw s1, 12(t0)

step_four:
    sw s5, 4(t0)
    continue_step_four:
        lw t1, 0(t0)
        beq t1, s2, continue_step_four

step_five:
    sw s4, 20(t0)
    continue_step_five:
        lw t1, 16(t0)
        beq t1, s2, continue_step_five

step_six:
    sw s6, 4(t0)
    sw s6, 20(t0) # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.6 ptos.)**: Se revisa el estado del robot. Si está apagado, se enciende antes de seguir. Luego, si está trabajando, se revisa su estado hasta verificar que terminó de trabajar.
- `step_two` **(0.6 ptos.)**: Se revisa el estado de la impresora. Si está apagada, se enciende antes de seguir. Luego, si está imprimiendo, se revisa su estado hasta verificar que terminó de imprimir.
- `step_three` **(0.4 ptos.)**: Se configura un total de 60 copias de impresión; se configura la impresión en blanco y negro; y el robot rellena las tintas de la impresora.
- `step_four` **(0.5 ptos.)**: Se inicia la impresión y se revisa el estado de la impresora hasta que termine de imprimir.
- `step_five` **(0.5 ptos.)**: Se le instruye al robot que apile las impresiones y se espera a que termine de apilarlas.
- `step_six` **(0.4 ptos.)**: Se apaga tanto la impresora como el robot y termina el programa.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.